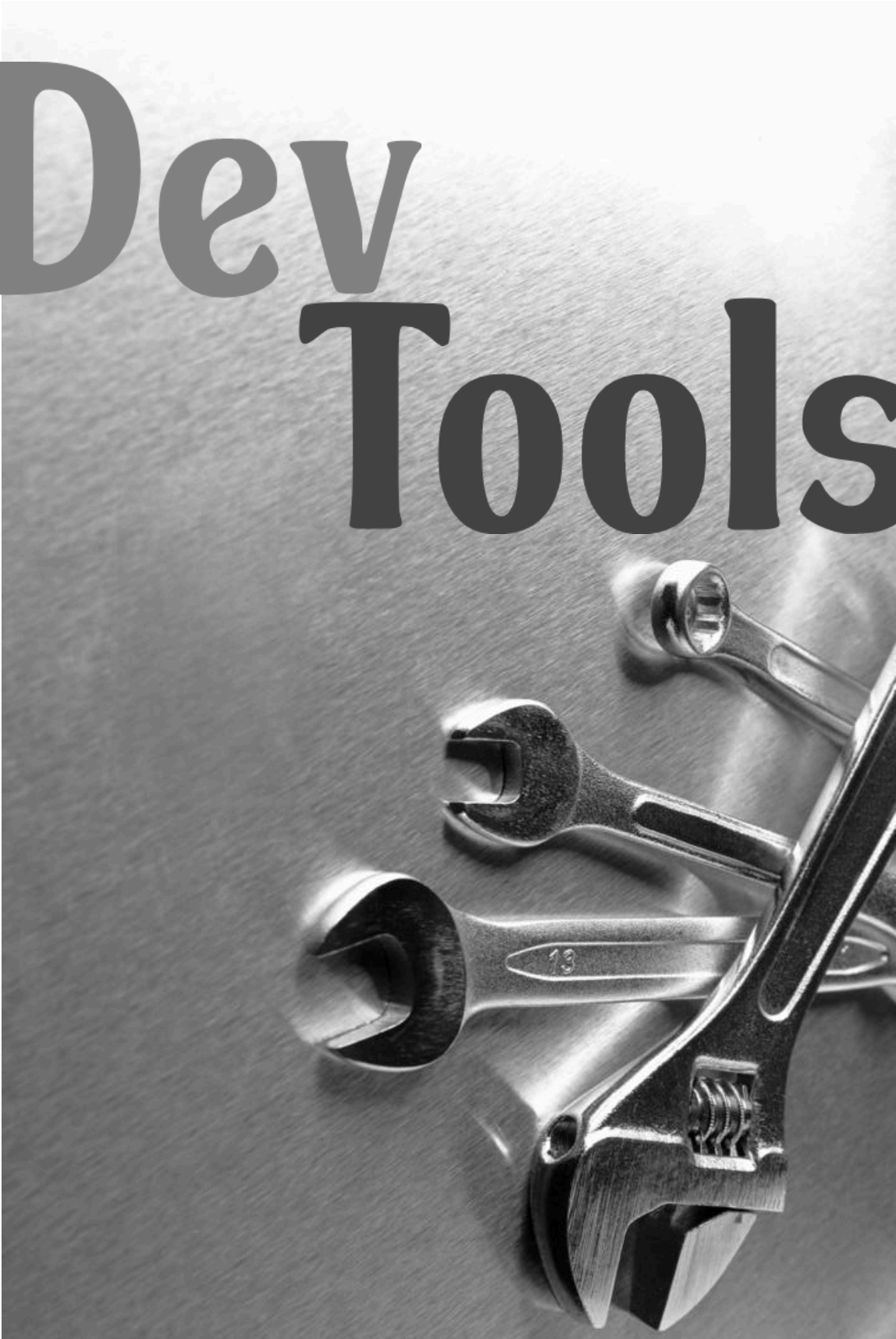


Dev Tools

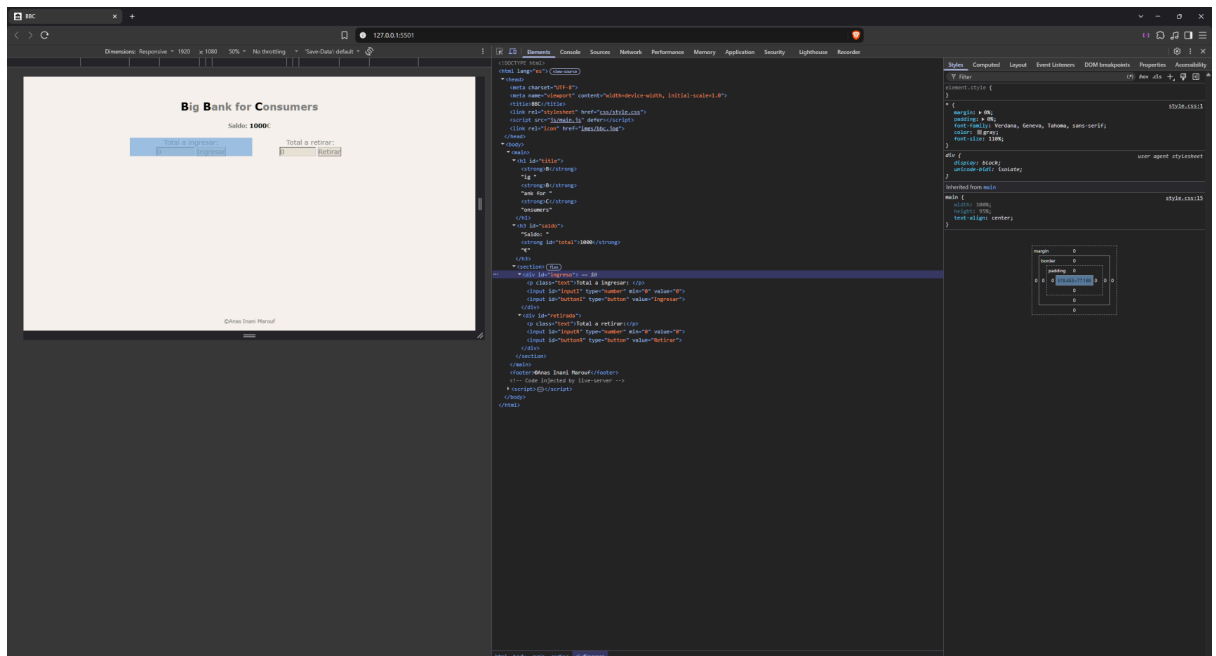


ÍNDICE

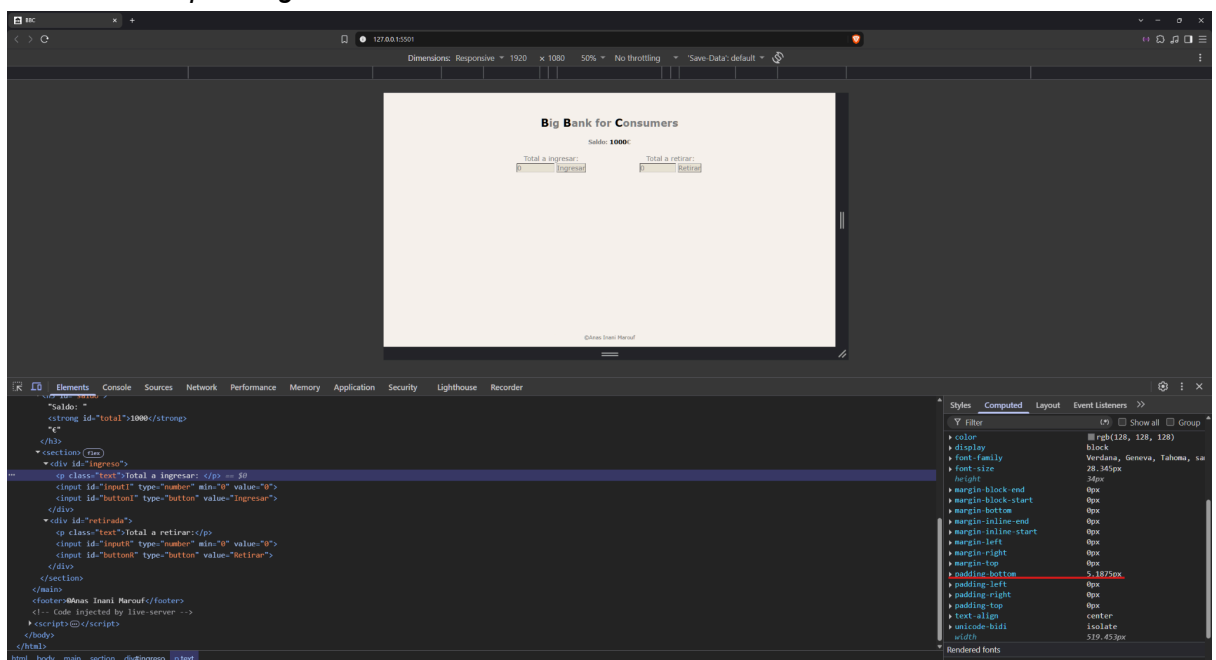
1. Elements / Inspector.....	3
2. Console.....	5
3. Sources / Depurador.....	7
4. Network.....	14
5. Application.....	15
6. Performance / Lighthouse (nivel básico).....	16

1. Elements / Inspector

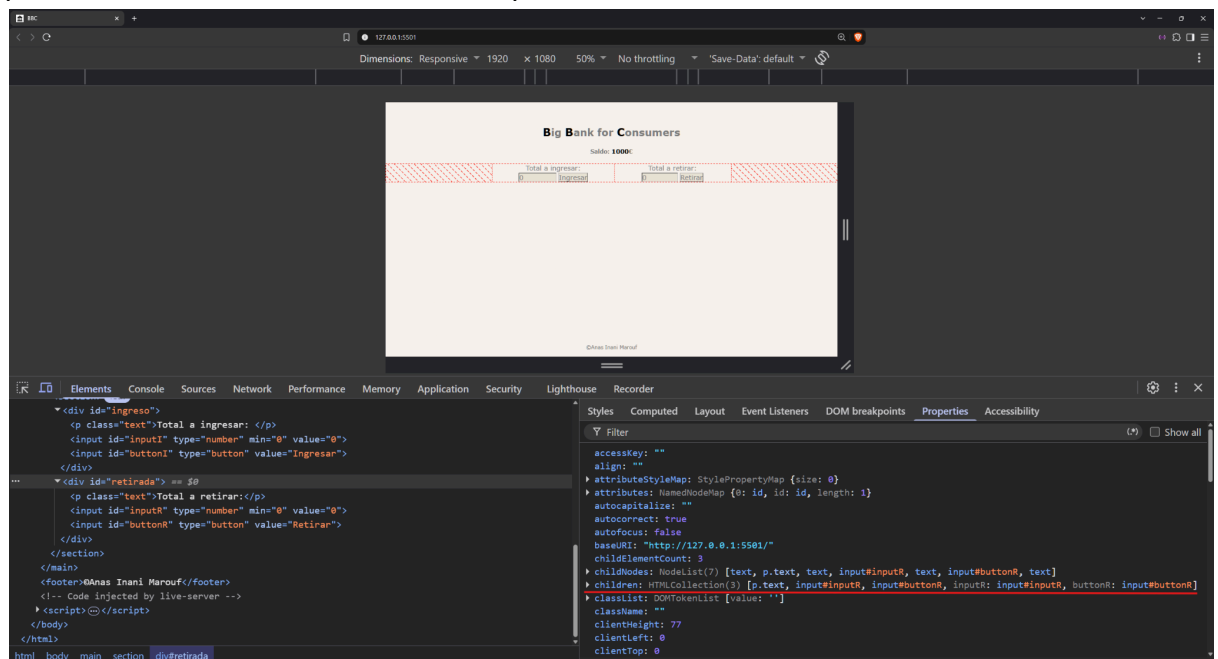
- En el apartado **styles**, si vas pulsando cada elemento de la pestaña elements, puedes ver que estilos tiene el apartado que esta seleccionado, en este caso, el div de ingreso, y ademas, puedes poner el cursor sobre el selector de elementos y te mostrara el tamaño que ocupa



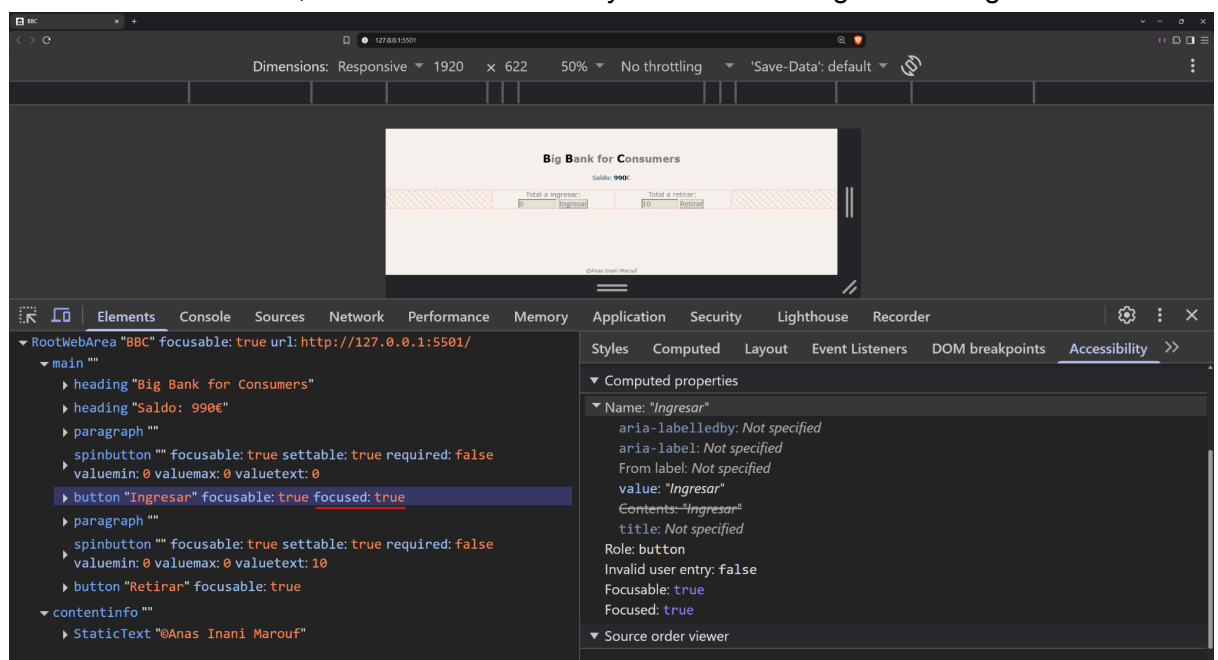
- En el apartado **computed**, se puede ver todos los estilos que afectan a un elemento en concreto, hayas puesto o no un valor, por ejemplo, en mi codigo css no he puesto ningun *margin*, sin embargo, en el panel computed me muestra el valor por defecto de ese elemento, que es 0px, si yo le hubiera dado un valor me lo mostraria, como es el caso del *padding-bottom*



- En el apartado **properties** se puede ver algunas propiedades de la pagina en general, ademas de todas las propiedades de la etiqueta DOM seleccionada, por ejemplo, el div con id retirada, desde la pestaña properties se pueden ver sus hijos, es decir los elementos que estan contenidos dentro del div. Y no solo se pueden ver que tipo de etiquetas son (p, input) si no que ademas justo al lado de cada una te pone el nombre del class o del id si es que tuviera



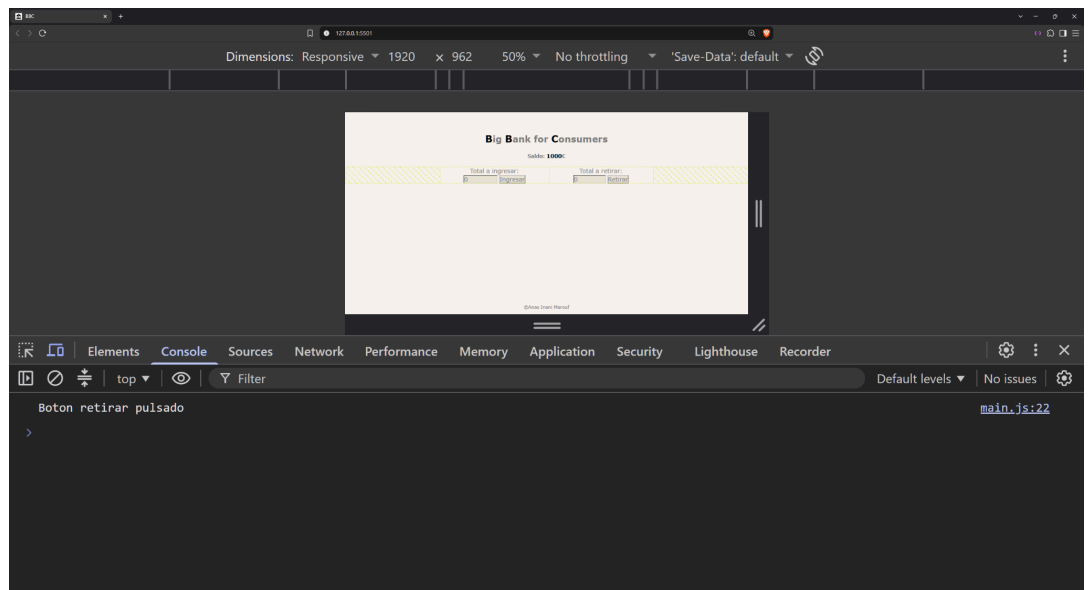
- En el apartado **accessibility**, si vas pulsando diferentes elementos puedes ver que estan recibiendo el foco, o sea que si pulsas el boton enter en el boton ingresar que esta siendo focuseado, actuara como un click y la cantidad designada se ingresara



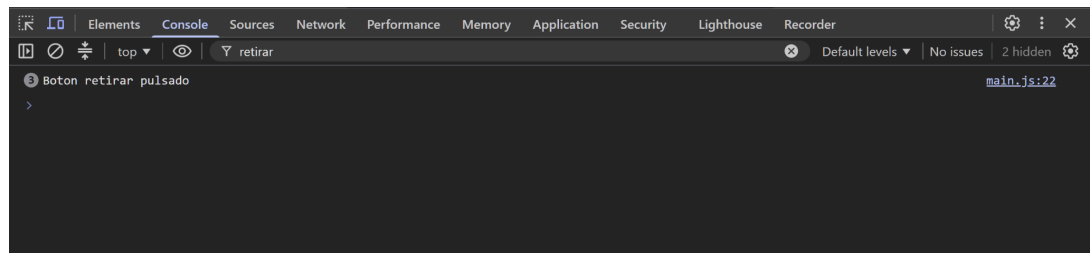
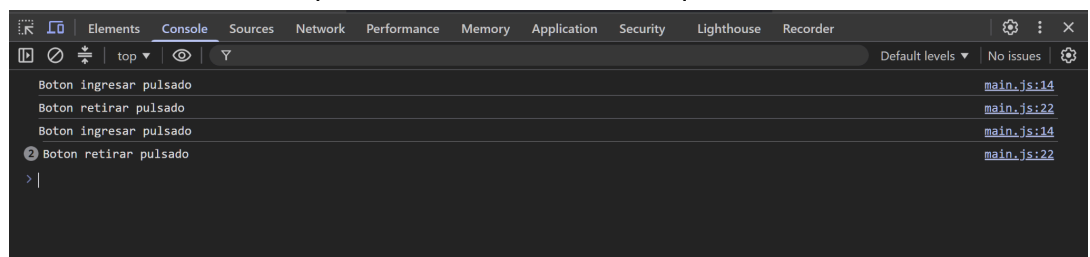
2. Console

- **console.log**: Al pulsar el boton retirar, se muestra por consola un mensaje para confirmar que el evento funciona

```
18  /**
19   * Evento que se ejecuta cuando se pulsa el boton retirar
20   */
21  bRetirar.addEventListener("click", () => {
22    console.log("Boton retirar pulsado");
23    retirar();
24  });
```

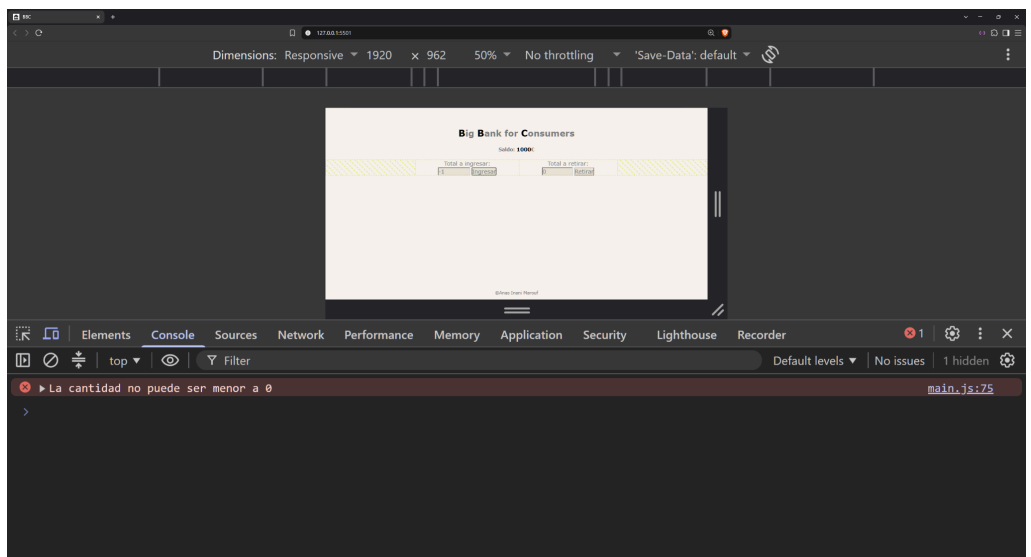


- **Filtrado de mensajes**: Al poner cualquier palabra o frase en el filtro, se mostrara unicamente aquellas lineas de la consola que coincidan

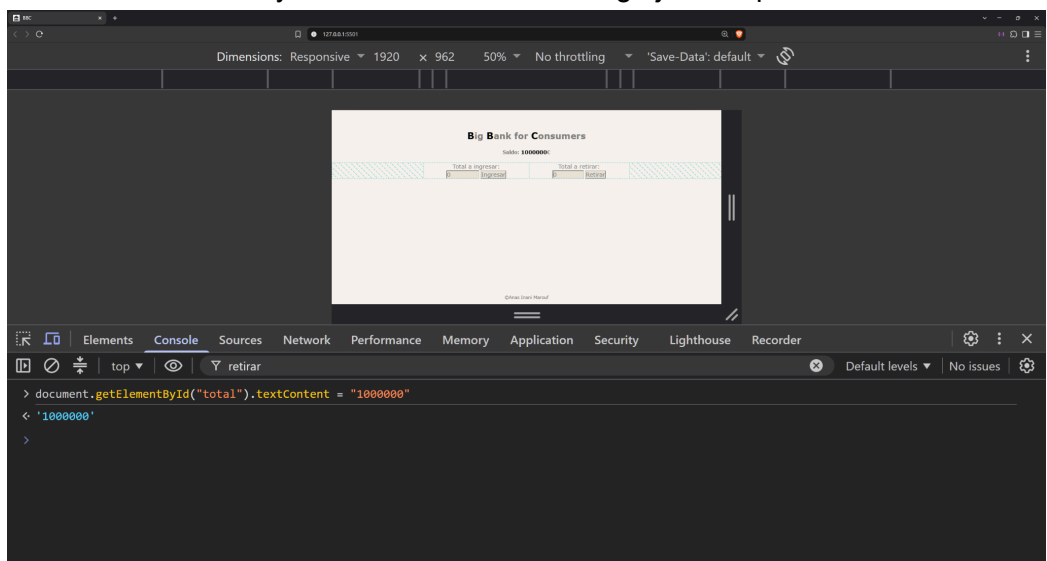


- **console.error:** Al pulsar un boton, si la cantidad puesta en el input correspondiente es menor a 0, se mostrara en consola un mensaje de error

```
59 /**
60  * Funcion que valida si se puede realizar el ingreso o la retirada,
61  * si todo esta correcto llama a la funcion actualizaSaldo()
62  */
63  * @param tipoInput Parametro tipo String, para saber sobre que
64  * input trabajar
65  * @param esRetirada Parametro tipo boolean, sirve para saber
66  * si actuar como un ingreso (false)
67  * o una retirada (true)
68  */
69  function validar(tipoInput, esRetirada) {
70
71      let cantidad = parseFloat(document.getElementById(tipoInput).value); //variable que obtiene la cantidad que el usuario ha puesto en el input
72
73      if (cantidad < 0) {
74          alert("La cantidad no puede ser menor a 0");
75          console.error("La cantidad no puede ser menor a 0");
76      }
77      } else { //cantidad mayor o igual a 0
78
79          if (esRetirada) { //Retirada
80
81              if (cantidad > saldo) {
82                  throw new Error("La cantidad retirada no puede ser mayor al saldo");
83              }
84              } else actualizaSaldo(-cantidad); //cantidad es menor que el saldo
85
86          } else actualizaSaldo(cantidad); //Ingreso
87      }
88  }
89  }
```

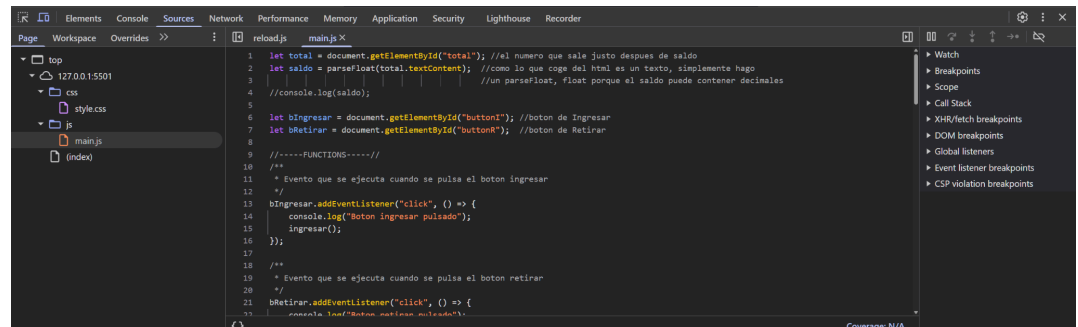


- **Ejecución de js directamente en la consola:** Se puede obtener acceso a una variable de html y cambiarla mediante codigo javascript en la consola

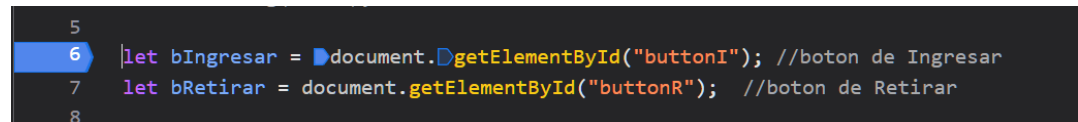


3. Sources / Depurador

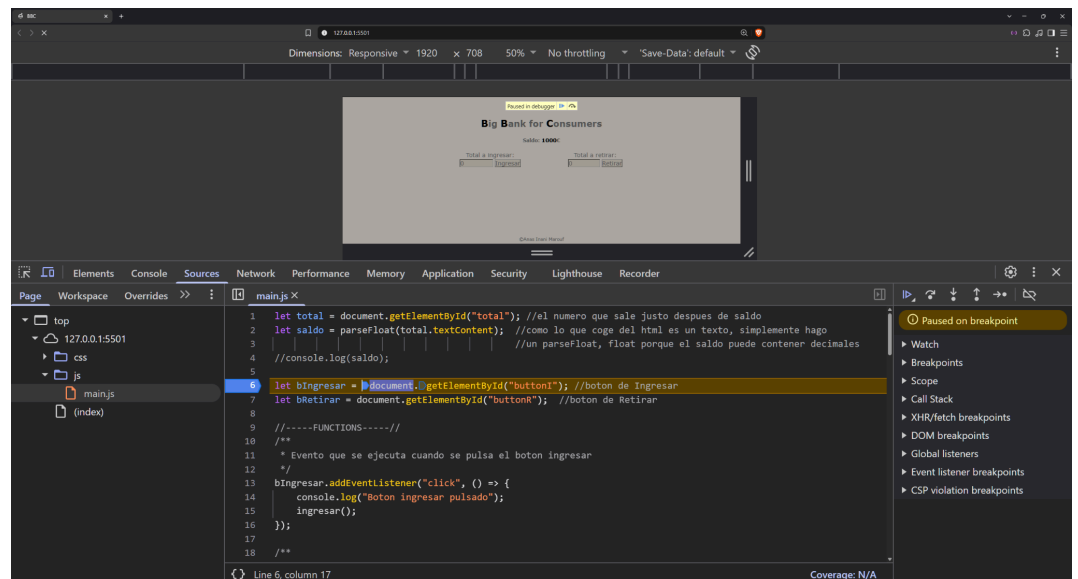
- Para **localizar y abrir un archivo .js** en el panel **Sources**, simplemente hay que irse al apartado **page** y abrir el archivo de dentro de la carpeta js, como esta en el proyecto



- **Añadir un breakpoint haciendo clic en el numero de la linea:** Te vas al numero de la linea a la que quieras aplicar un punto de ruptura y le das una vez con el click



Asi cuando recargues la pagina, no se ejecutara todo el codigo hasta que hayan pasado todos los puntos



- **Diferencia entre Step over, Step into, Step out y Resume:**
 - **Step over:** Ejecuta la siguiente línea del código, pero si se encuentra con una llamada a una función se la salta por completo

This screenshot shows the Chrome DevTools Sources panel with the 'main.js' file open. The 'Step Over' button (a blue play icon with a right arrow) is highlighted in the bottom toolbar. The current execution position is at line 15, column 5, where the `ingresar();` function is called. The status bar at the bottom indicates 'Line 15, column 5' and 'Coverage: N/A'. The file content shows two event listeners: one for 'buttonI' (line 13) and one for 'buttonR' (line 23). The 'buttonI' listener calls `ingresar();` at line 15.

```

6 let bIngresar = document.getElementById("buttonI"); //boton
7 let bRetirar = document.getElementById("buttonR"); //boton
8
9 //-----FUNCTIONS-----//
10 /**
11  * Evento que se ejecuta cuando se pulsa el boton ingresar
12  */
13 bIngresar.addEventListener("click", () => {
14   console.log("Boton ingresar pulsado");
15   ingresar();
16 });
17
18 /**
19  * Evento que se ejecuta cuando se pulsa el boton retirar
20  */
21 bRetirar.addEventListener("click", () => {
22   console.log("Boton retirar pulsado");
23   retirar();
24 });
25
26 /**
27  * Funcion que va actualizando el numero del saldo a medida

```

This screenshot shows the Chrome DevTools Sources panel with the 'main.js' file open. The 'Step Over' button is highlighted in the bottom toolbar. The current execution position is at line 37, column 5, where the `saldo += cantidad;` line is executed. The status bar at the bottom indicates 'Line 37, column 5' and 'Coverage: N/A'. The file content shows a function `actualizaSaldo(cantidad)` that updates the balance and a function `ingresar()` that calls `validar()`.

```

26 /**
27  * Funcion que va actualizando el numero del saldo a medida
28  * que se vaya ingresando o retirando cierta cantidad
29  *
30  * @param cantidad La cantidad que se sumara al saldo (puede
31  * ser positiva o negativa, si es un ingreso entonces le lle
32  * una cantidad positiva, por lo cual se sumara al saldo,
33  * pero si es retirada le llegara una cantidad negativa, as
34  * se restara esa cantidad al saldo total)
35  */
36 function actualizaSaldo(cantidad) { cantidad = 0
37   saldo += cantidad;
38   total.innerText = saldo;
39 }
40
41 /**
42  * Funcion que llama a otra funcion validar(), y le envia el
43  * del input de texto para mas tarde obtener la cantidad que
44  * usuario quiera ingresar
45  */
46 function ingresar() {
47   validar("inputI", false);
48 }

```

- **Step into:** Ejecuta la siguiente línea del código, y a diferencia del step over, si hay una llamada a una función se mete dentro de ella y se ejecuta también línea por línea

```
4 //console.log(saldo);
5
6 let bIngresar = document.getElementById("buttonI"); //boton
7 let bRetirar = document.getElementById("buttonR"); //boton
8
9 //-----FUNCTIONS-----//
10 /**
11  * Evento que se ejecuta cuando se pulsa el boton ingresar
12  */
13 bIngresar.addEventListener("click", () => {
14     console.log("Boton ingresar pulsado");
15     ingresar();
16 });
17
18 /**
19  * Evento que se ejecuta cuando se pulsa el boton retirar
20  */
21 bRetirar.addEventListener("click", () => {
22     console.log("Boton retirar pulsado");
23     retirar();
24 });
25
26 /**
```

```
36 function actualizaSaldo(cantidad) {
37     saldo += cantidad;
38     total.innerText = saldo;
39 }
40
41 /**
42  * Funcion que llama a otra funcion validar(), y le envia e:
43  * del input de texto para mas tarde obtener la cantidad que
44  * usuario quiera ingresar
45  */
46 function ingresar() {
47     validar("inputI", false);
48 }
49
50 /**
51  * Funcion que llama a otra funcion validar(), y le envia e:
52  * del input de texto para mas tarde obtener la cantidad que
53  * usuario quiera retirar
54  */
55 function retirar() {
56     validar("inputR", true)
57 }
58
```

- **Step out:** Cuando se esta dentro de una funcion simplemente se sale a la siguiente linea del codigo (que no sea una funcion)

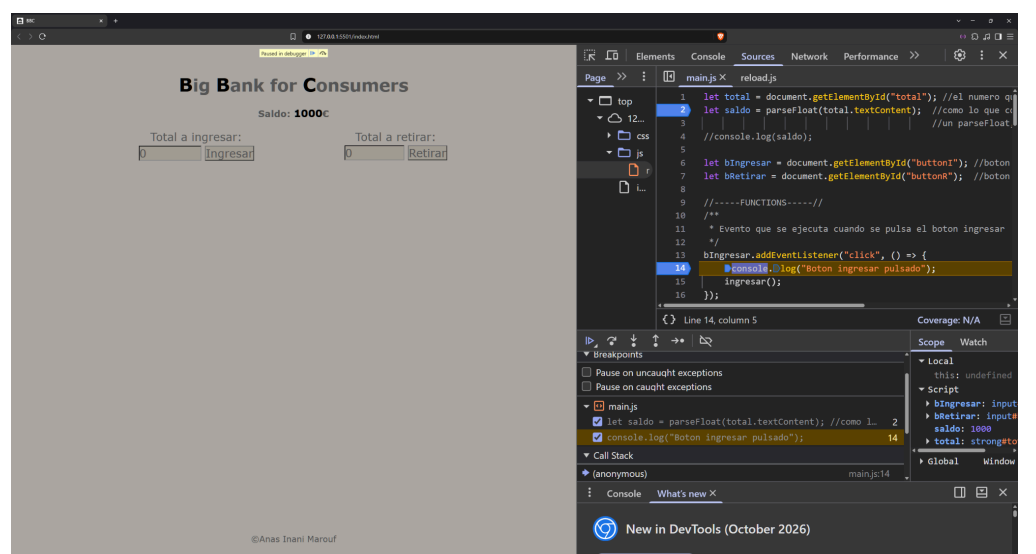
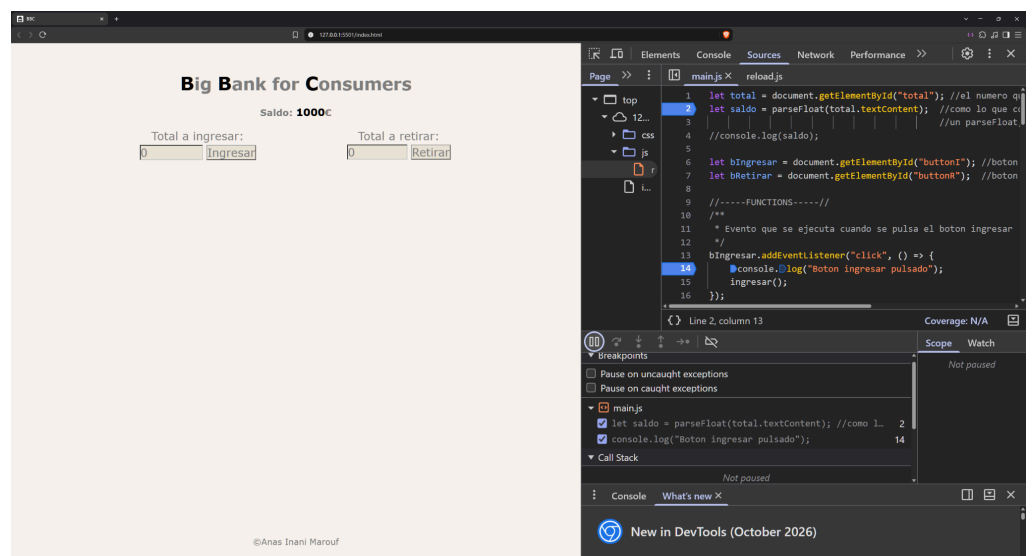
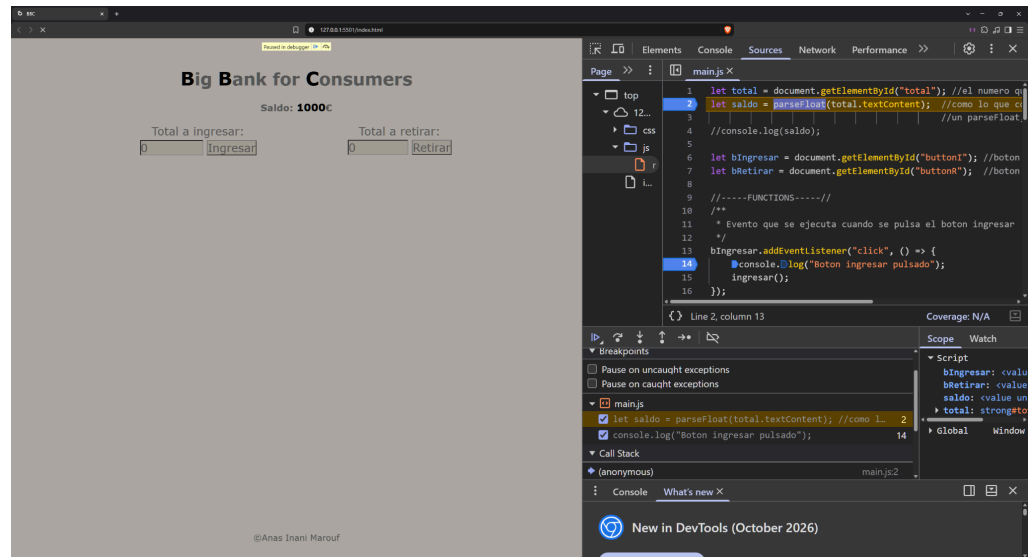
```
28  * que se vaya ingresando o retirando cierta cantidad
29  *
30  * @param cantidad La cantidad que se sumara al saldo (puede
31  * ser positiva o negativa, si es un ingreso entonces le lle
32  * una cantidad positiva, por lo cual se sumara al saldo,
33  * pero si es retirada le llegara una cantidad negativa, asi
34  * se restara esa cantidad al saldo total)
35  */
36  function actualizarSaldo(cantidad) { cantidad = 0
37  | saldo += cantidad;
38  | total.innerText = saldo;
39  | }
40
41  /**
42  * Funcion que llama a otra funcion validar(), y le envia el
43  * del input de texto para mas tarde obtener la cantidad que
44  * usuario quiera ingresar
45  */
46  function ingresar() {
47  |   validar("input1", false);
48  | }
49
```

{ } Line 37, column 5 Coverage: N/A

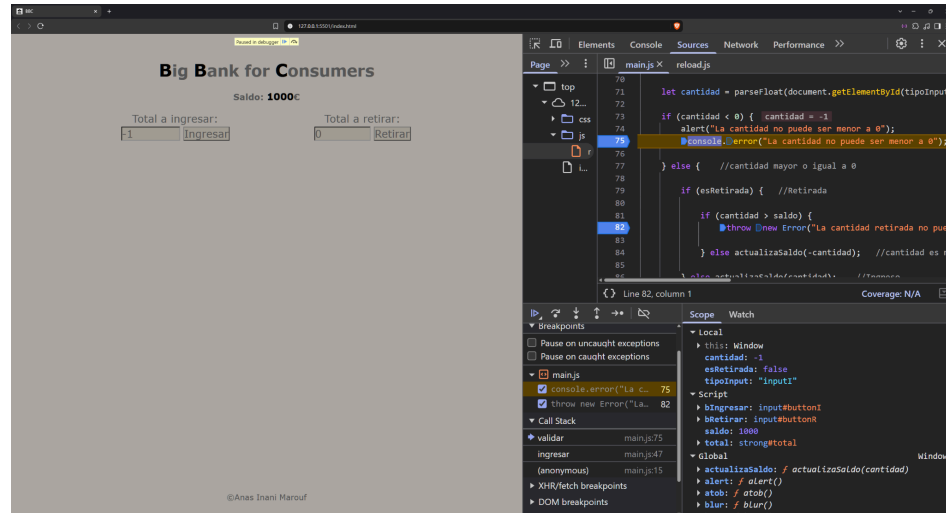
```
78  } else { //cantidad mayor o igual a 0
79  |
80  |   if (esRetirada) { //Retirada esRetirada = false
81  |   |   if (cantidad > saldo) { cantidad = 0
82  |   |   |   throw new Error("La cantidad retirada no pue
83  |   |   } else actualizarSaldo(-cantidad); //cantidad e
84  |   |   } else actualizarSaldo(cantidad); //Ingreso cant
85  |   }
86  | }
87  |
88  | }
89
```

{ } Line 88, column 1 Coverage: N/A

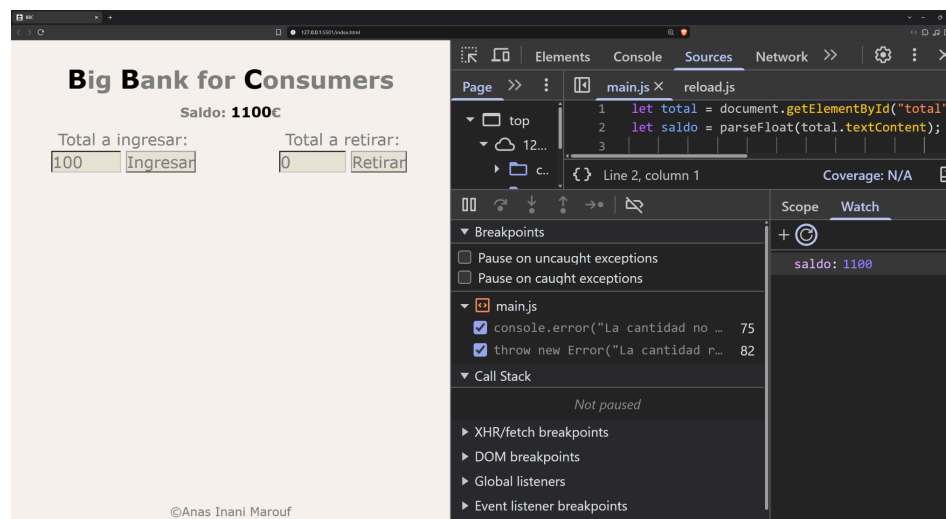
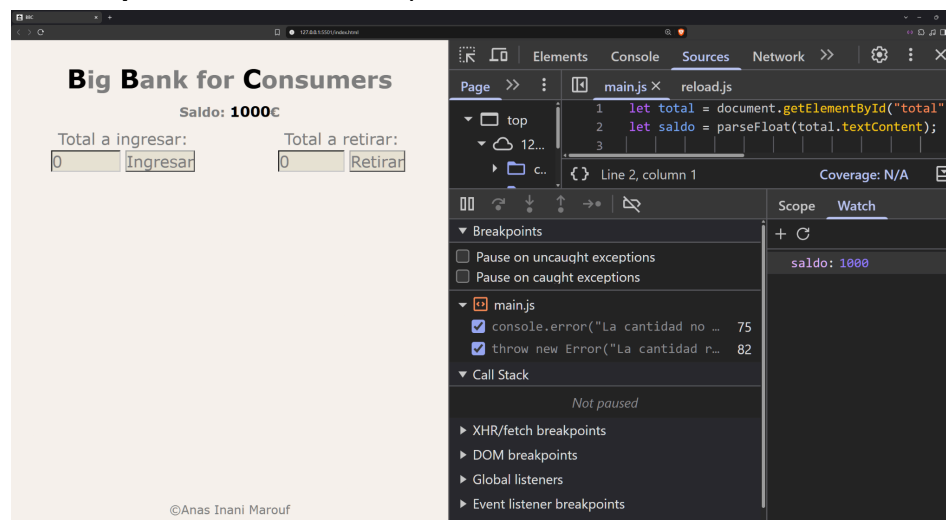
- **Resume:** Reanuda la aplicacion, y cuando se encuentre con otro punto de ruptura lo vuelve a parar hasta que se pulse otra vez el boton de resume



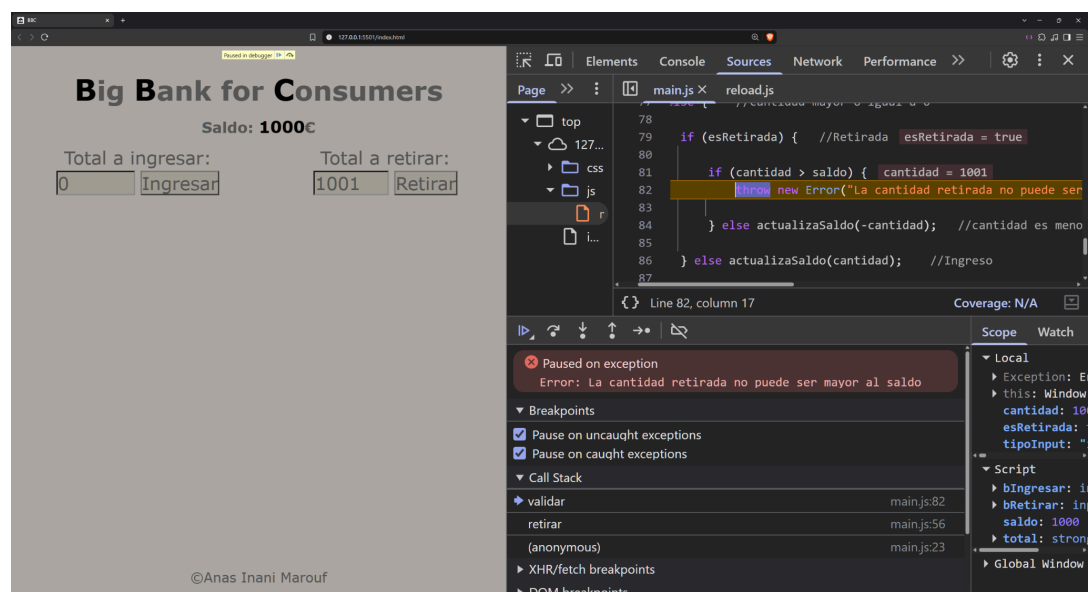
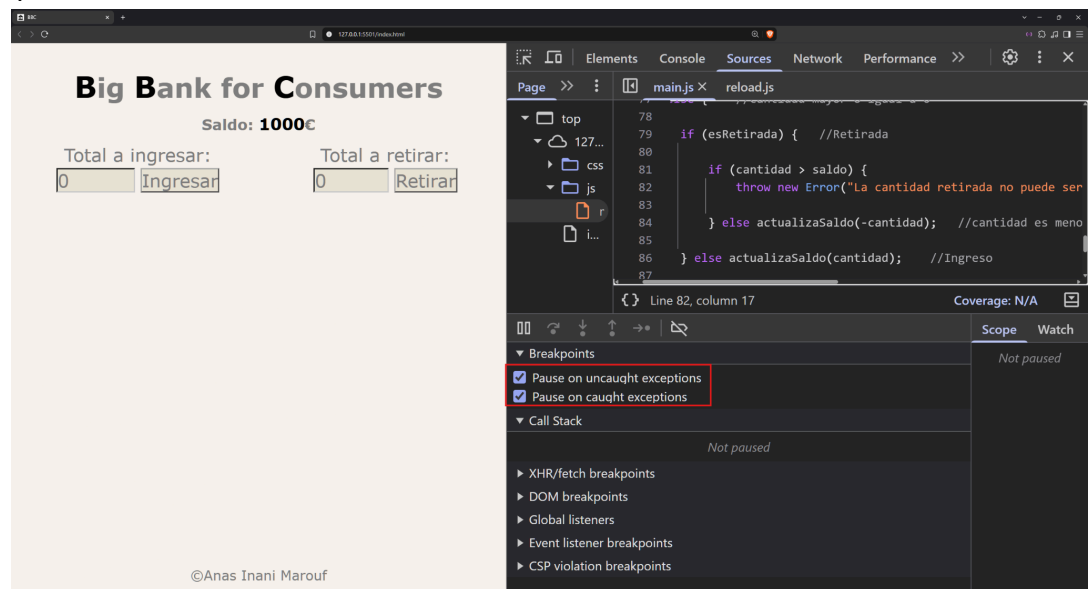
- **Uso del panel Scope:** Cuando pulso el boton ingresar y ve que es un numero negativo, en el pequeño panel de scope se enseñan las variables que estan siendo en uso, por ejemplo, se puede ver que la cantidad que se ha puesto es -1, y por eso ha saltado un error, porque no puede ser un numero negativo



- **Uso del panel Watch:** Se usa para monitorear el valor de una variable



- **Pause on exceptions:** Hay dos opciones, una es para los errores que tu hayas controlado y otra para los que no, y lo que hace es pausar la ejecución de la página dependiendo de lo que este seleccionado y si se produce algún error. Aquí por ejemplo aunque no tenga ningún breakpoint en el error que he controlado, la página se me pausa automáticamente, porque he seleccionado que sea así



4. Network

- En la pestaña **network** se puede ver el tiempo de carga de toda la pagina total, pero andemas se pueden ver cuanto tiempo han tardado cada uno de los archivos en cargarse

The screenshot shows a web browser window with the URL `127.0.0.1:5501/index.html`. The page title is "Big Bank for Consumers" and the balance is "Saldo: 1000C". There are two input fields for "Total a ingresar:" and "Total a retirar:" with buttons "Ingresar" and "Retirar" respectively. The footer says "©Anas Inani Marouf".

The Chrome DevTools Network tab is open, showing a list of resources. The table below represents the data shown in the Network tab:

Name	Status	Type	Initiator	Size	Time
index.html	304	document	Other	0.2 kB	2 ms
style.css	304	stylesheet	index.html:7	0.2 kB	1 ms
main.js	304	script	index.html:8	0.2 kB	2 ms
ws	101	websocket	index.html:51	0.0 kB	Pending
bbc.jpg	304	jpeg	Other	0.2 kB	1 ms

Summary: 5 requests | 969 B transferred | 13.9 kB resources | Finish: 22 ms | DOMContentLoaded: 17 ms

- Además se puede seleccionar un cierto rango para ver que es lo que se ha cargado en ese espacio de tiempo

The screenshot shows the same web browser window and DevTools Network tab. The time range is set to 20 ms to 40 ms. The table below represents the filtered data:

Name	Status	Type	Initiator	Size	Time
ws	101	websocket	index.html:51	0.0 kB	Pending
bbc.jpg	304	jpeg	Other	0.2 kB	1 ms

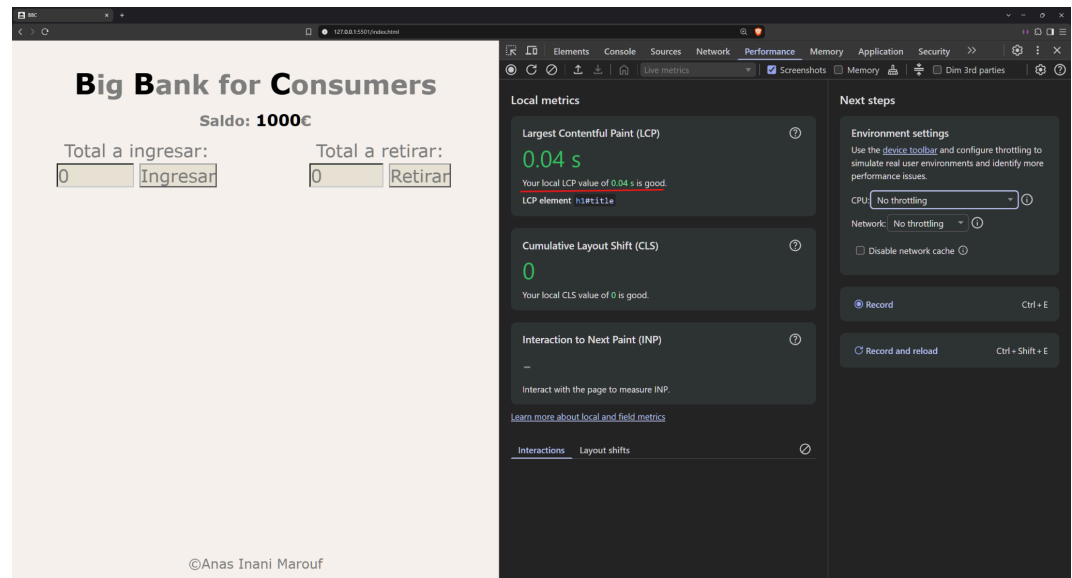
Summary: 2 / 5 requests | 0.2 kB / 1.0 kB transferred | 7.6 kB / 13.9 kB resources | Finish: 22 ms | DOMContentLoaded: 17 ms

5. Application

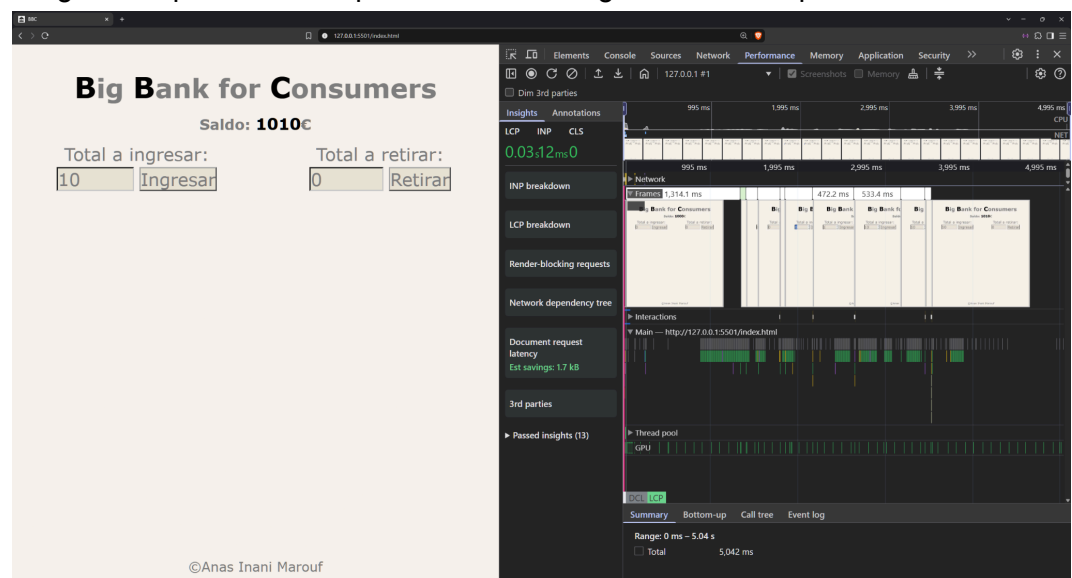
- **LocalStorage:** Se utiliza para guardar algunos datos para utilizarse al volver a abrir la pagina despues de cerrarla, lo malo de este metodo es que si no lo borra el programador mediante alguna pieza de codigo, se va a quedar en el ordenador o movil del usuario ocupando muhco espacio, a no ser que el mismo usuario lo borre manualmente
- **SessionStorage:** Lo mismo que el localStorage, pero la unica diferencia esque si cierras la pestaña estos datos se borran, por lo que no tienes que preocuparte que el usuario tenga basura en su movil constantemente
- **Cookies:** Sirven para guardar datos, pero a diferencia de las dos anteriores, esta se comunica con el servidor y ocupan muy poco espacio en tu ordenador, ademas le puedes poner una fecha de expiracion

6. Performance / Lighthouse (nivel básico)

- En la pestaña **performance**, se puede ver si el tiempo que tarda en cargar la pagina es buena o no



- Además, hay una opción para recargar la página y grabar el tiempo que tarda en ejecutarse un evento, y aparece un apartado a la derecha que te dice cuánto tiempo ha tardado desde que se ha iniciado la página hasta después de haberse terminado de ejecutar el evento, y se pueden ver incluso los fotogramas que han ido capturándose a lo largo de este tiempo



- En la pestaña **lighthouse**, se puede ver el rendimiento general y la accesibilidad de la web, en mi caso, en cuanto a accesibilidad me lo ha puntuado en un 75/100, mostrandome que puedo mejorar el tamaño de los botones, los inputs, etc, ya que a algunos usuarios de moviles les podria llegar a costar acceder a los distintos apartados de la web. Tambien te va dirigiendo por los distintos apartados que te ha puntuado, y diciendote mejoras que podrias llegar a implementar, etc. (Este apartado es un poco complejo pero me ha gustado mucho)

